ABC by Example and Experiments (Top-Down Approach)

Focus on Control Flow …

… but also a general overview

```
@ <stdio.hdr>

fn main()
{
    printf("hello, world!\n");
}
```

```
@ <stdio.hdr>

fn main()
{
    printf("hello, world!\n");
    printf("hello, world!\n");
    printf("hello, world!\n");
}
```

```
@ <stdio.hdr>

fn main()
{
    printf("hello, world! 1\n");
    printf("hello, world! 2\n");
    printf("hello, world! 3\n");
}
```

```
@ <stdio.hdr>

fn foo()
{
    printf("I am foo\n");
}

fn main()
{
    printf("hello, world! 1\n");
    foo();
    printf("hello, world! 2\n");
    foo();
    printf("hello, world! 3\n");
}
```

```
@ <stdio.hdr>

fn main()
{
    printf("hello, world! 1\n");
    foo();
    printf("hello, world! 2\n");
    foo();
    printf("hello, world! 3\n");
}

fn foo()
{
    printf("I am foo\n");
}
```

```
@ <stdio.hdr>

fn foo();

fn main()
{
    printf("hello, world! 1\n");
    foo();
    printf("hello, world! 2\n");
    foo();
    printf("hello, world! 3\n");
}

fn foo()
{
    printf("I am foo\n");
}
```

```
@ <stdio.hdr>

fn foo(val: int);

fn main()
{
    printf("hello, world! 1\n");
    foo(1);
    printf("hello, world! 2\n");
    foo(2);
    printf("hello, world! 3\n");
}

fn foo(val: int)
{
    printf("I am foo. val = %d\n", val);
}
```

```
extern fn printf(fmt: -> char, ...): int;

fn foo(val: int);

fn main()
{
    printf("hello, world! 1\n");
    foo(1);
    printf("hello, world! 2\n");
    foo(2);
    printf("hello, world! 3\n");
}

fn foo(val: int)
{
    printf("I am foo. val = %d\n", val);
}
```

```
@ <stdio.hdr>

fn foo(val: int): int
{
    return val * 2;
}

fn main()
{
    printf("foo(%d) = %d\n", 1, foo(1));
    printf("foo(%d) = %d\n", 2, foo(2));
}
```

```
@ <stdio.hdr>

fn foo(val: int): int
{
    if (val > 1) {
        return val * 2;
    } else {
        return 42;
    }
}

fn main()
{
    printf("foo(%d) = %d\n", 1, foo(1));
    printf("foo(%d) = %d\n", 2, foo(2));
}
```

```
@ <stdio.hdr>

fn foo(val: int): int
{
    if (val > 1) {
        val = val * 2;
    } else {
        val = 42;
    }
    return val;
}

fn main()
{
    printf("foo(%d) = %d\n", 1, foo(1));
    printf("foo(%d) = %d\n", 2, foo(2));
}
```

```
@ <stdio.hdr>

fn foo(val: int): int
{
    if (val > 1) {
        val = val * foo(val - 1);
    } else {
        val = 1;
    }
    return val;
}

fn main()
{
    printf("foo(%d) = %d\n", 0, foo(0));
    printf("foo(%d) = %d\n", 5, foo(5));
}
```

```
@ <stdio.hdr>

fn foo(val: int): int
{
    while (val < 10) {
        val = val * 3;
    }
    return val;
}

fn main()
{
    printf("foo(%d) = %d\n", 1, foo(1));
    printf("foo(%d) = %d\n", 2, foo(2));
}
```

- Formalization of things …
    - Syntax of the ABC language
    - Semantics
    - …
- Technical foundation
    - Memory
    - …